

National Emission Standards for Hazardous Air Pollutants (NESHAP) Maximum Achievable Control Technology (MACT) Floor Analysis for Example Industry Units for Final Rule

OAR-OCAP-NRD-EDAB or author names here

October 31, 2025

```
#> Compiling rjags model...
#> Starting 3 rjags simulations using a PSOCK cluster with 3 nodes on host
#> 'localhost'
#> Simulation complete
#> Finished running the simulation
#> Compiling rjags model...
#> Starting 3 rjags simulations using a PSOCK cluster with 3 nodes on host
#> 'localhost'
#> Simulation complete
#> Finished running the simulation
```

Introduction

Section 112 of the Clean Air Act (CAA) requires the the U.S. Environmental Protection Agency (EPA) establish National Emission Standards for Hazardous Air Pollutants (NESHAP) for the control of the hazardous air pollutants (HAP) emitted from both new and existing sources in a source category. These standards must reflect the maximum degree of reduction in emissions that is achievable. The minimum level of control is referred to as the “Maximum Achievable Control Technology (MACT) floor.” The method for determining the MACT floor as the minimum control level allowed for establishing standards for a NESHAP is defined for both new and existing sources by CAA Section 112(d)(3). For new sources, the MACT floor cannot be less stringent than the emission control that is achieved in practice by the best-controlled similar source. For existing sources, the MACT floor cannot be less stringent than the average emission limitation achieved by the best-performing 12 % of existing sources for source categories with 30 or more sources, or the best-performing 5 sources for source categories with fewer than 30 sources.

The purpose of this memorandum is to present the methodology and the results of the MACT floor analysis for the NESHAP source category Example Industry. This MACT floor analysis uses data collected in a nationwide survey of Example Industry unit owners and operators conducted by the EPA in 2025 under the Information Collection Request for NESHAP for Example Industry.

MACT Floor Analysis Hazardous Air Pollutants

The specific chemicals, compounds, or groups of compounds designated to be HAP are listed in *CAA section 112(b)*. From this HAP list, the following HAP, among others, were identified to be emitted from Example Industry units and were included in the MACT floor analysis: *chemical X*, *chemical Y*, *chemical Z*...

MACT Floor Analysis subcategories

Under *CAA section 112(d)(1)*, the EPA has the discretion to “...distinguish among classes, types, and sizes of sources within a category or subcategory in establishing...” standards. When separate subcategories are

established, a MACT floor is determined separately for each subcategory. To determine whether the EGU source category warrants subcategorization for the MACT floor analysis, the EPA reviewed Example Industry design, operating information, and air emissions data compiled in the ICR data set and other information collected by the Agency for development of the NESHAP for this source category. Based on this review, the EPA concluded that there are significant design and operational differences in Example Industry that affect the levels of emissions of certain HAP to justify subcategorizing Example Industry for the purpose of establishing MACT floors.

For the MACT floor analysis, the following Example Industry subcategories were defined:

- **Subcategory 1.** *describe it here and how/why it was determined*
- **Subcategory 2.** *describe it here and how/why it was determined*

Calculating Emission Values used in the MACT Floor Analysis

The EPA received emissions data from industry respondents under the Example Industry MACT ICR in a number of different units of measure and converted these units of measure to a common *insert measurement units*.

If any outlier analysis or subsequent QA/QC was done on the emissions data prior to MACT floor analysis, please include it here.

MACT Floor Analysis Methodology

The first step in the MACT floor analysis for each subcategory and HAP (or surrogate) was to rank all of the Example Industry units in the subcategory (for which there were emissions data in the ICR data set) for the pollutant type by emission level (lowest to highest). From this ranking, a MACT floor pool of sources was identified for determining the minimum control level allowed for the MACT floor, consistent with the criteria defined for new and existing sources by CAA Section 112(d)(3). For the new-source MACT floors, the best-controlled similar source was identified for which there were individual source test run data in the ICR data set. The selection of top performing sources for the existing source MACT floor was performed using the `MACT_EG()` function in the `UPLforOAR` R package version 0.1.0, with details in Appendix C. The selection of the best performing source for the new source MACT floor was performed using the `MACT_NSPS()` function in the `UPLforOAR` R package version 0.1.0, with details in appendix C.

For the existing-source MACT floors, selection of the MACT floor pool size (i.e., number of units to be included in the determination of the average emission limitation value) was determined on an individual subcategory basis as described below.

- **Subcategory 1.** *do any subcategories need different handling?*
- **Subcategory 2.** *do any subcategories need different handling?*

Probability distribution Selection

The next step in the MACT floor analysis was to incorporate data variability into the calculations of the applicable MACT floor limits for the subcategories using a 99% upper prediction limit (UPL) approach. Specifically, the UPL is calculated as the 99th quantile of the average of 3 future emission test values. The level of confidence represents the level of protection afforded to facilities whose emissions are in line with the best performers, and consequently, the level of confidence is not arbitrary. For example, a 99% level of confidence means that a facility whose emissions are in line with the best performers has one chance in 100 of exceeding the floor limit. A prediction interval for a single future observation (or an average of several test observations) is an interval that will, with a specified degree of confidence, contain the next (or the average of some other pre-specified number) of randomly selected observation(s) from a population. In other words, the UPL estimates what the upper bound of future values will be, based upon present or past background samples taken. The UPL consequently represents the value at which we can expect the mean of future observations for the HAP or HAP surrogate emissions to fall within a specified level of confidence,

based upon the measurements from an independent sample from the same population. The UPL approach encompasses all the data point-to-data point variability.

Implementation

The form of the UPL equation differs depending upon the nature of the data set to which it is applied. To this end, the data sets were evaluated for each HAP and HAP surrogate to ascertain whether the data were normally, log-normally, skew-normally, or gamma distributed. For each distribution, a probability density function was fit to the test run emissions using Bayesian Markov Chain Monte Carlo (MCMC). This analysis was implemented in R using `runjags` and JAGS (Just Another Gibbs Sampler) software, with three MCMC chains with different initial values, completely uninformed priors, a burn-in and adapt of 10,000 iterations each, and a final set of 10,000 iterations retained in posterior distributions. Random seeds were specified to control for stochasticity and ensure reproducibility. The JAGS model scripts were defined using the function `write_likelihood()`, and executed using functions `setup_likelihood()` and `run_likelihood()` in the UPLforOAR R package version 0.1.0. See Appendix C for more details.

Convergence was assessed quantitatively by checking Gelman-Rubin diagnostics and qualitatively by viewing plots of the iteration chains to ensure they were well-mixed and without trend, as well as histograms of the posterior distributions to ensure they converged. These checks were carried out using the functions `converge_likelihood()` and `converge_figs()` in the UPLforOAR R package version 0.1.0. See Appendix C for more details.

Both quantitative and qualitative evaluations were used to select the appropriate distribution. For each distribution, the Sum of Squared Errors (*SSE*) is calculated as:

$$SSE = \sum (y_i - \hat{y}_i)^2$$

Where y_i is the empirical test run probability density at emission value x_i and $\hat{y}_i$ is the predicted observation probability density at emission value x_i . The lower the *SSE*, the better the probability distribution represents the probability of the emission values. The empirical test run probability densities, y_i , were also checked if they were significantly different than the predicted probability densities, $\hat{y}_i$, by comparing them to the 95% confidence interval (CI) around the predicted densities derived from posterior distributions. The uncertainty around the predicted probability density can be variable; thus one can have a very different probability at the same x_i but be within the 95% CI if the uncertainty is very large, or have a significantly different probability at the same x_i even though the y_i and $\hat{y}_i$ are extremely similar if there is very little uncertainty. The more y_i that are statistically equivalent to $\hat{y}_i$, the better the distribution represents the probability of x_i emission values. A proper probability density function will integrate to 1, so if the integral of the fitted distribution is substantially different from 1 this can indicate an issue. Lastly, a visual assessment comparing the empirical and predicted probability densities across the range of possible emission values can identify any problematic behavior or differentiate between distributions. These fit metrics were calculated out using the functions `fit_likelihood()` and `obs_density()` in the UPLforOAR R package version 0.1.0. See Appendix C for more details.

To calculate the UPL, which is the 99th quantile of the average of 3 future emission test values, the posterior distributions of 3 new draws from the fitted probability distribution were averaged and then the UPL is determined as the 99th quantile of this resulting distribution. The fitted distribution is truncated by lower and upper bounds of 0 and $3 \times \max(x_i)$ emissions, which prevents extreme values from long-tailed distributions biasing the UPL and otherwise has no effect on the results. The 2.5th and 97.5th quantiles of the posterior distributions of the predicted probability density $\hat{y}_i$ at each x_i test run value were used to calculate 95% CI for the goodness-of-fit metrics. The median of the posterior distributions of the predicted probability density $\hat{y}_i$ were used in comparisons to y_i to calculate the *SSE*. The median of the posterior distributions for all $\hat{y}_i$ across the possible range of emission test run values was monitored for use in plotting the distributions and visual comparisons to the empirical distributions. All of these computations were done using the function `output_likelihood()` in the UPLforOAR R package version 0.1.0. See Appendix C for more details.

The parameter estimation and UPL calculations for each distribution investigated were performed using the function `Bayesian_UPL()` in the `UPLforOAR` R package version 0.1.0, which wraps the functions described above and concatenates the results for easy comparison. See Appendix C for more details.

The gamma distribution parameter prior distributions, MCMC initial chain values, and Gelman-Rubin diagnostics for convergence that were used in the *chemical X subcategory A* existing source UPL calculation are listed below.

Table 1: Parameter information for the gamma distribution UPL calculation for existing source chemical X, subcategory A.

	Prior Parameterdistribution	Chain 1 initial value	Chain 2 initial value	Chain 3 initial value	Gelman-Rubin diagnostic	Converged?
Rate	~dunif (0, 43.9)	3.920	5.880	1.960	1	Yes
Shape	~dunif (0, 75.6)	0.602	0.301	0.903	1	Yes

The skew-normal distribution parameter prior distributions, MCMC initial chain values, and Gelman-Rubin diagnostics for convergence that were used in the *chemical X subcategory A* new source UPL calculation are listed below.

Table 2: Parameter information for the skew-normal distribution UPL calculation for new source chemical X, subcategory A.

	Prior Parameterdistribution	Chain 1 initial value	Chain 2 initial value	Chain 3 initial value	Gelman-Rubin diagnostic	Converged?
Omega	~dunif (0, 1.72)	0.00846	0.01270	0.00423	1.00	Yes
Xi	~dnorm (0, 0.00581)	0.00232	0.00116	0.00348	1.04	Yes
Alpha	~dunif (-100, 100)	1.64000	0.81900	2.46000	1.03	Yes

UPL Calculation for Normal Data

In the case of normally distributed data, the MACT floor limit is an UPL calculated as the 99th quantile of the average of 3 new emission runs, where the probability density, y_i , of normally distributed emissions, x_i , is dependent on parameters *mean*, μ , and *standard deviation*, σ :

$$y_i = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x_i - \mu)^2}{2\sigma^2}}$$

UPL Calculation for Log-normal Data

In the case of log-normally distributed data, the MACT floor limit is an UPL calculated as the 99th quantile of the average of 3 new emission runs, where the probability density, y_i , of log-normally distributed emissions, x_i , is dependent on parameters μ , the mean of $\log(x_i)$ and σ , the standard deviation of $\log(x_i)$:

$$y_i = \frac{1}{x_i\sqrt{2\pi\sigma^2}} e^{-\frac{(\log(x_i) - \mu)^2}{2\sigma^2}}$$

UPL Calculation for Skew-normal Data

In the case of skew-normally distributed data, the MACT floor limit is an UPL calculated as the 99th quantile of the average of 3 new emission runs. The Skew-normal distribution is defined by three parameters; location ξ , scale ω , and skewness α . This is equivalent to a Normal distribution when $\alpha = 0$, it is right-skewed if $\alpha > 0$, and left-skewed if $\alpha < 0$. The probability density, y_i , of Skewed distributed emissions is:

$$y_i = \frac{2}{\omega} \phi\left(\frac{x_i - \xi}{\omega}\right) \Phi\left(\alpha \left(\frac{x_i - \xi}{\omega}\right)\right)$$

Where $\phi(x)$ is the standard normal probability distribution, $\Phi(x)$ is its cumulative probability distribution function, and x_i is transformed to $\frac{x_i - \xi}{\omega}$. Using the error function, this can be expanded to:

$$y_i = \frac{2}{\omega} \frac{1}{\sqrt{2\pi}} e^{-\frac{\left(\frac{x_i - \xi}{\omega}\right)^2}{2}} \frac{1}{2} \left[1 + \frac{2}{\sqrt{\pi}} \int_0^{\alpha \frac{x_i - \xi}{\omega\sqrt{2}}} e^{-t^2} dt \right]$$

UPL Calculation for Gamma Data

In the case of gamma distributed data, the MACT floor limit is an UPL calculated as the 99th quantile of the average of 3 new emission runs, where the probability density, y_i , of gamma distributed emissions, x_i , is dependent on parameters *shape*, α , and *rate*, β :

$$y_i = \frac{\beta^\alpha}{\Gamma(\alpha)} x_i^{\alpha-1} e^{-\beta x_i}$$

Note that Γ in this case is the gamma *function* not the Gamma probability distribution.

UPL Calculation for Beta Data

In the case of beta distributed data, the MACT floor limit is an UPL calculated as the 99th quantile of the average of 3 new emission runs, where the probability density, y_i , of beta distributed emissions, x_i , is dependent on parameters α and β :

$$Y_i = \frac{\Gamma(\alpha + \beta)}{\Gamma(\alpha)\Gamma(\beta)} x_i^{\alpha-1} (1 - x_i)^{\beta-1}$$

Note that Γ in this case is the gamma *function* not the Gamma probability distribution.

Adjustment for Below Detection Level Emissions Data

To account for the effect of measurement imprecision associated with the data in the ICR databases (that include method detection level data), a protocol specified by the EPA was used. The procedure for determining a representative detection level (RDL) begins with identifying all of the available reported pollutant-specific method detection levels for the best performing units regardless of any subcategory (e.g., existing or new, fuel type, etc.). From that combined pool of data, we calculate the arithmetic mean value. By limiting the data set to those tests used to establish the floor or emissions limit (i.e., best performers), the EPA believes that the result is representative also of the best performing testing companies and laboratories. The EPA also believes that the outcome should minimize the effect of a test(s) with an inordinately high method detection level (e.g., the sample volume was too small, the laboratory technique was insufficiently sensitive, or the procedure for determining the detection level was other than that specified). The EPA then calls the resulting mean of the method detection levels the RDL, and the Agency considers it characteristic of accepted source emissions measurement performance.

The second step in the process is to calculate three times the RDL to compare with the calculated floor or emissions limit. We use the multiplication factor of 3. For comparing to the floor, if three times the RDL

were less than the calculated floor or emissions limit (e.g., calculated from the UPL), we would conclude that measurement variability was adequately addressed. The calculated floor or emissions limit would need no adjustment. If, on the other hand, the value equal to three times the RDL were greater than the UPL, we would conclude that the calculated floor or emissions limit does not account entirely for measurement variability. If indicated, we substituted the value equal to three times the RDL for the calculated MACT floor UPL emissions level.

Add any RDL content specific to your source category

Table 3: Here is an example of a table in case you need one

	Existing MACT floor (old units)	Existing MACT floor (new units)	New MACT floor
Subcategory 1	HAP 1, HAP 2	HAP 1, HAP 2	HAP 1, HAP 2, X, Y, Z
Subcategory 2	HAP 1	HAP 1	HAP 1, X, Y, Z
Subcategory 3	HAP 1, HAP 2, HAP 3	HAP 1, HAP 2, HAP 3	HAP 1, HAP 2, HAP 3, X, Y, Z
Subcategory 4	None	None	None
Subcategory 5	None	None	X, Y, Z

MACT Floor Data Set

describe any data that was excluded here and why

Tables with the full data sets used for each existing source MACT floor analysis can be found listed in Appendix A. Tables with the full data sets used for each new source MACT floor analysis can be found listed in Appendix B.

The final data set for the *chemical X* existing source MACT floor for *subcategory A* included 24 test runs from 5 sources. The best performer for the new source calculation was A with an average emission of 0.01004 from 3 test runs.

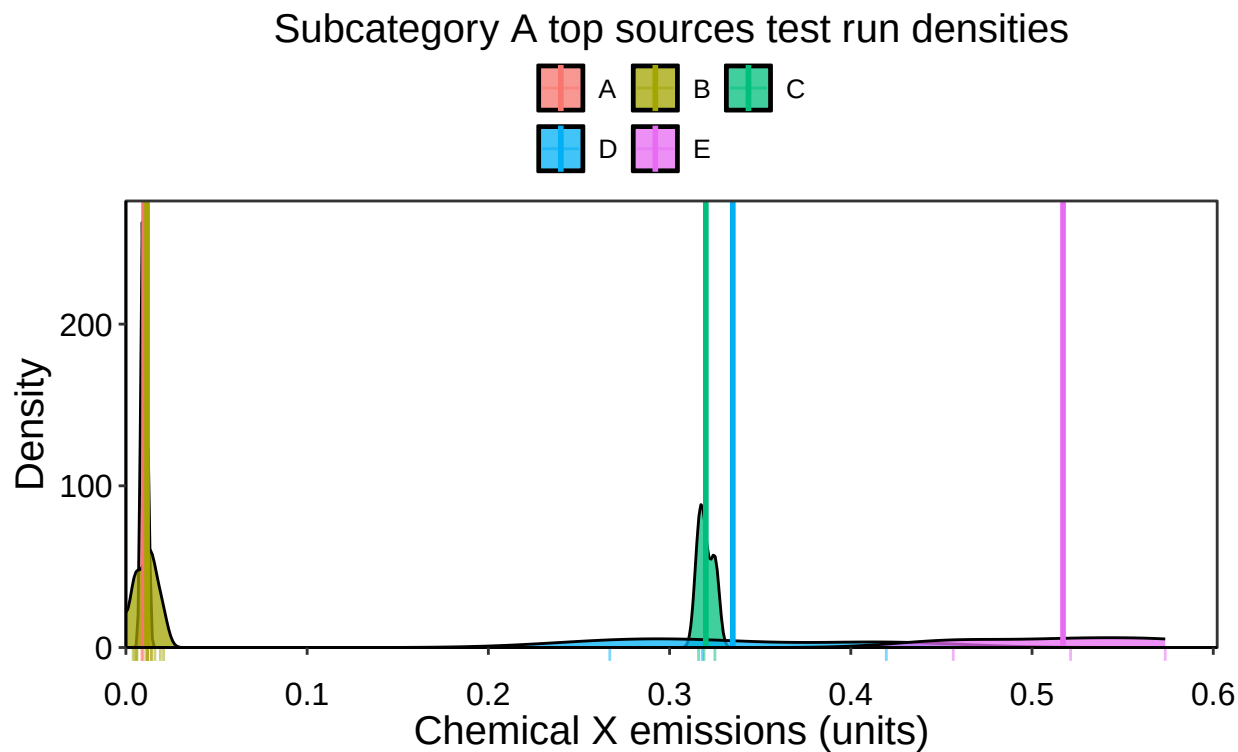


Figure 1: Observation density of emissions by source type. Source averages are indicated by vertical lines.

Figures with density plots of emission test run data were created using `ggplot2` in R. The density curves were generated using `obs_density()` in `UPLforOAR` version 0.1.0, using a `gamma` kernel type, a zero lower bound, a bandwidth of $\sigma n^{-2/5}$ where σ is the emission value standard deviation and n is the number of runs. For further details see the R code in Appendix C.

MACT Floor Analysis Results

Existing Source Chemical X in Subcategory A

The distribution that best represents the existing source data set for *chemical X* in *subcategory A* is the gamma distribution, with a corresponding UPL of 0.6086 (*insert measurement units*). The gamma distribution was the best fit distribution with a *SSE* of 40.93, 15 emission values within the 95% CI of predictions, and the probability density function integrated to approximately 0.9.

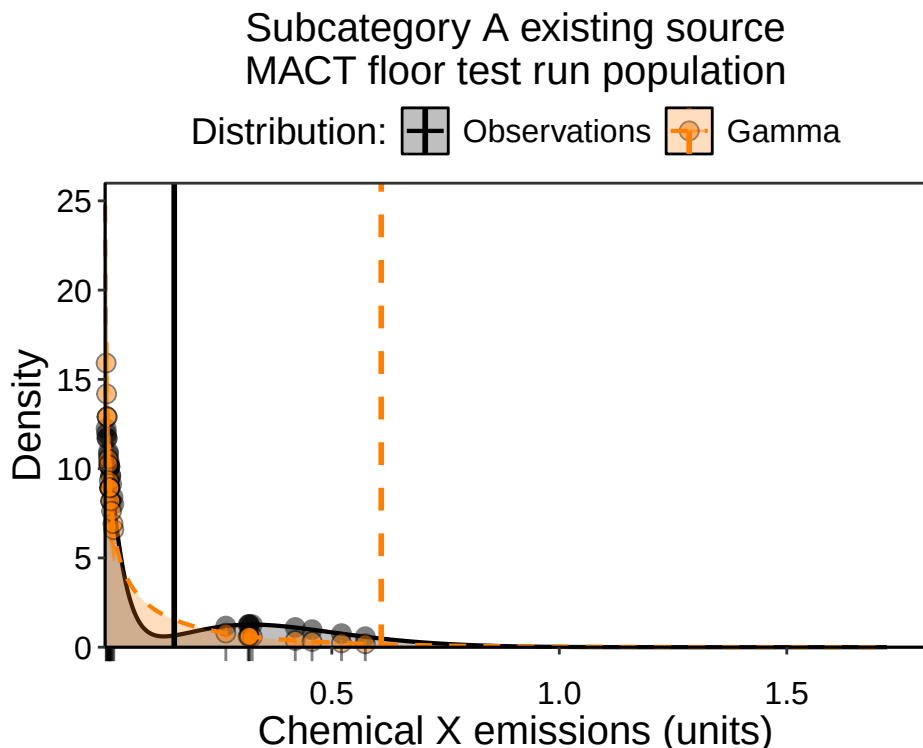


Figure 2: Observation density of chemical X emissions for the overall population of test runs for the existing source MACT floor for subcategory A. The observation data are indicated in black as points and a rug along the axis, with the observation density distribution as a black line. The fitted distribution that is the basis of the UPL estimate is colored orange. The average of the emission test runs is the vertical black line and the UPL result is the vertical orange line.

The gamma distributions parameters converge well, passing both Gelman-Rubin diagnostic checks and with well-mixed MCMC chains (left figures below) and an approximately normal posterior distributions around the converged value (right figures below).

```
#> Compiling rjags model...
#> Starting 3 rjags simulations using a PSOCK cluster with 3 nodes on host
#> 'localhost'
#> Simulation complete
#> Finished running the simulation
#> [[1]]
```

Gamma

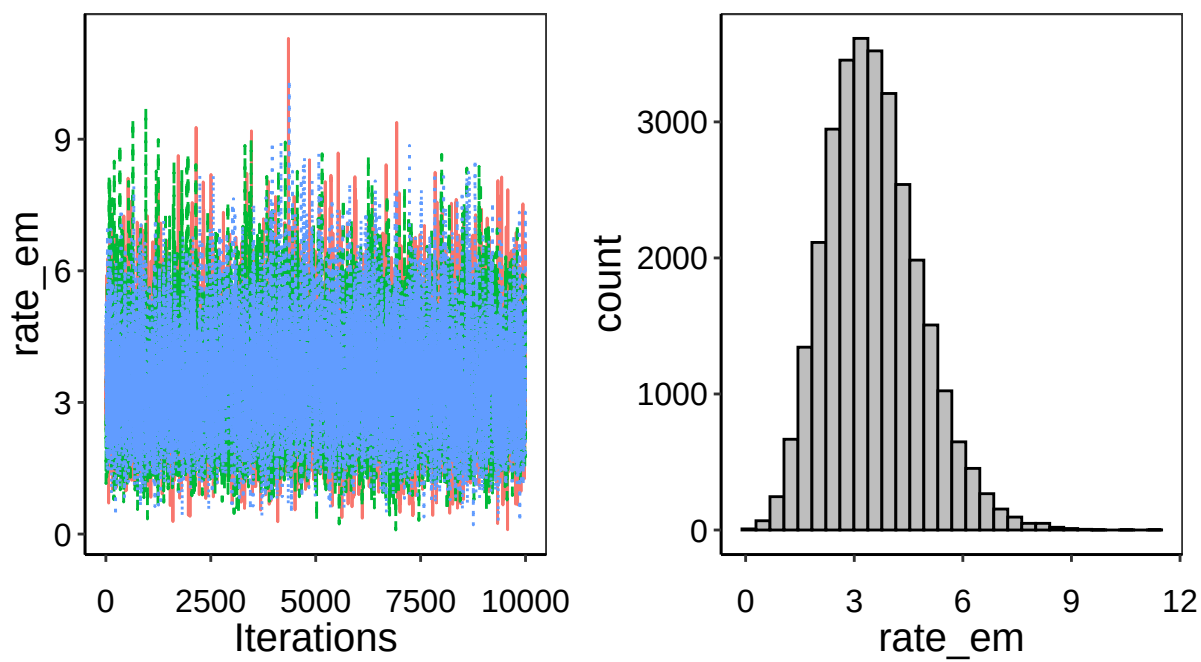


Figure 3: Convergence figures for gamma parameters for existing source MACT floor analysis for chemical X in subcategory A.

```
#>  
#> [[2]]
```

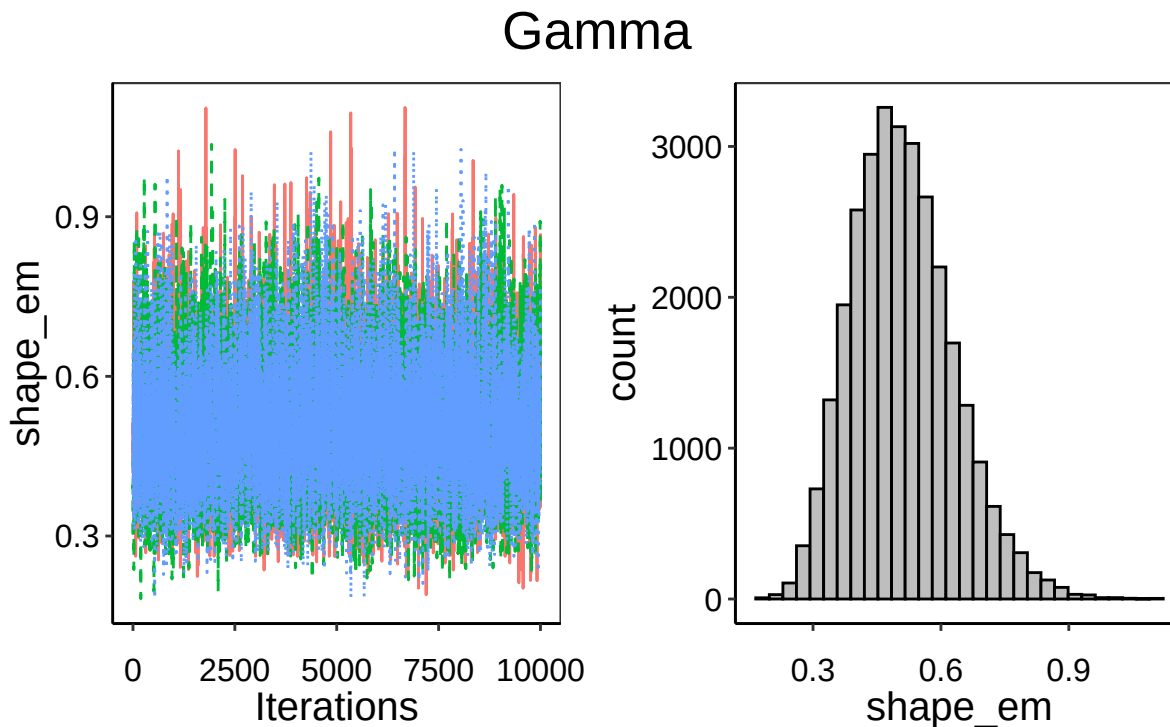


Figure 4: Convergence figures for gamma parameters for existing source MACT floor analysis for chemical X in subcategory A.

New Source Chemical X in Subcategory A

The distribution that best represents the new source data set for *chemical X* in *subcategory A* is the skew-normal distribution, with a corresponding UPL of 0.01577 (*insert measurement units*). The skew-normal distribution was the best fit distribution with a *SSE* of 630.8, 3 emission values within the 95% CI of predictions, and the probability density function integrated to approximately 0.65.

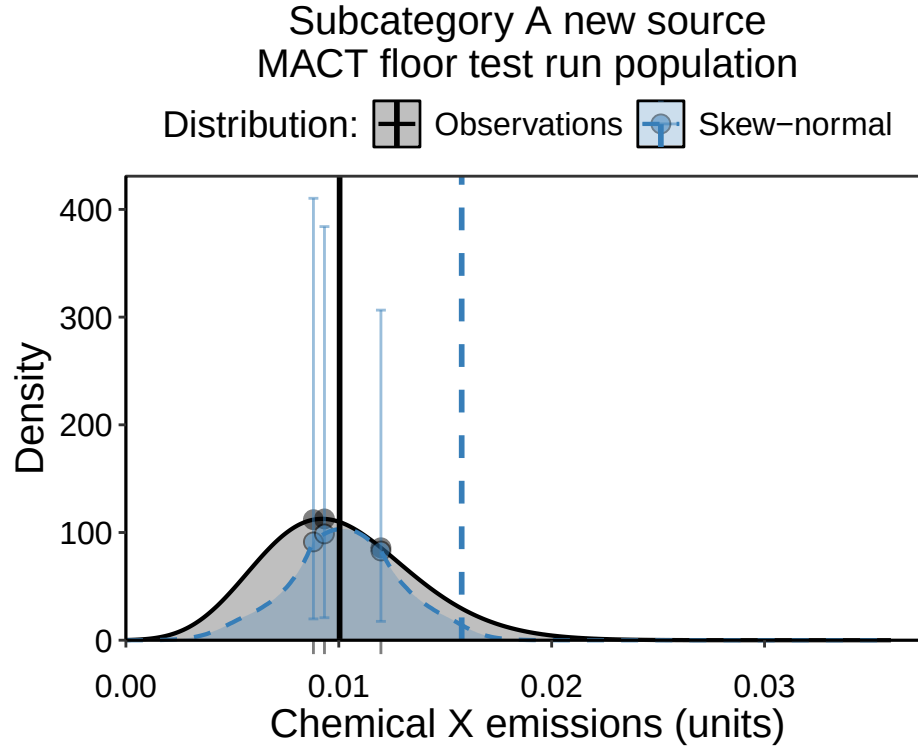


Figure 5: Observation density of chemical X emissions for the overall population of test runs for the new source MACT floor for subcategory A. The observation data are indicated in black as points and a rug along the axis, with the observation density distribution as a black line. The fitted distribution that is the basis of the UPL estimate is colored blue. The average of the emission test runs is the vertical black line and the UPL result is the vertical blue line.

The gamma distributions parameters converge well, passing both Gelman-Rubin diagnostic checks and with well-mixed MCMC chains (left figures below) and an approximately normal posterior distributions around the converged value (right figures below).

```
#> Compiling rjags model...
#> Starting 3 rjags simulations using a PSOCK cluster with 3 nodes on host
#> 'localhost'
#> Simulation complete
#> Finished running the simulation
#> [[1]]
```

Skewed

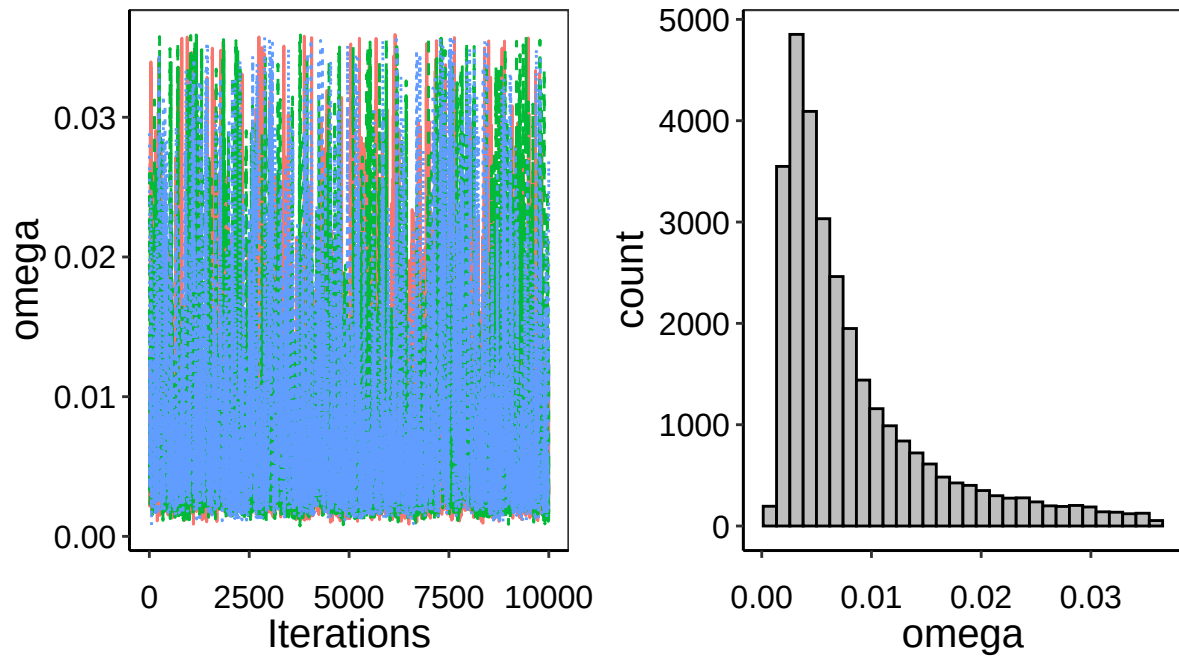


Figure 6: Convergence figures for skew-normal parameters for new source MACT floor analysis for chemical X in subcategory A.

```
#>  
#> [[2]]
```

Skewed

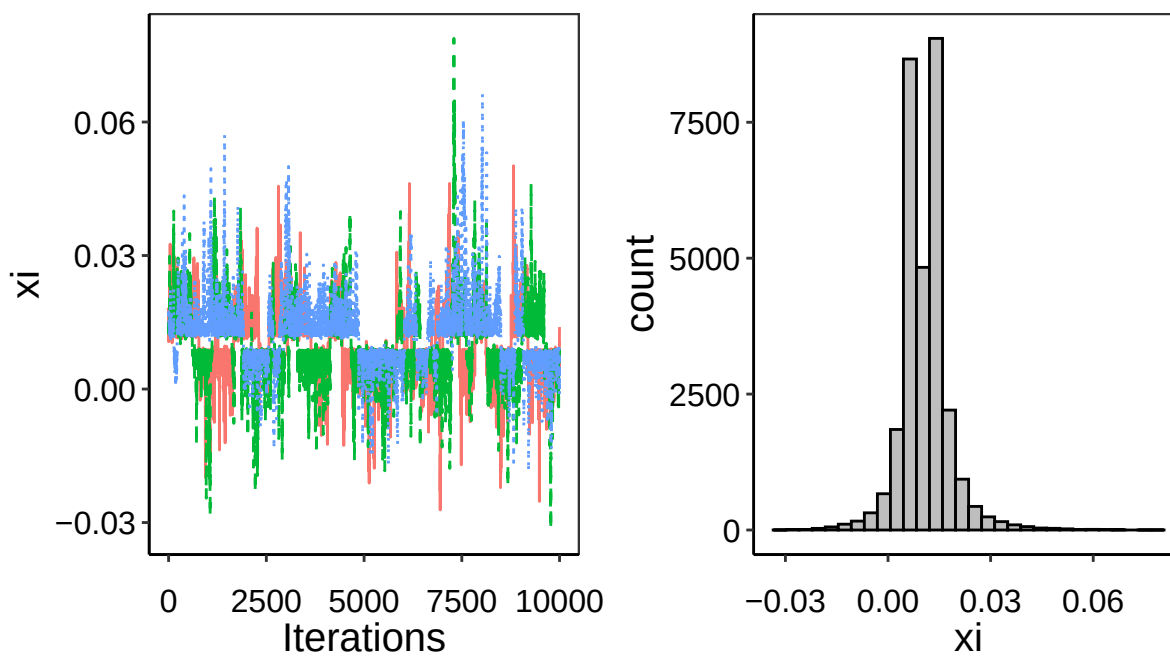


Figure 7: Convergence figures for skew-normal parameters for new source MACT floor analysis for chemical X in subcategory A.

```
#>
#> [[3]]
```

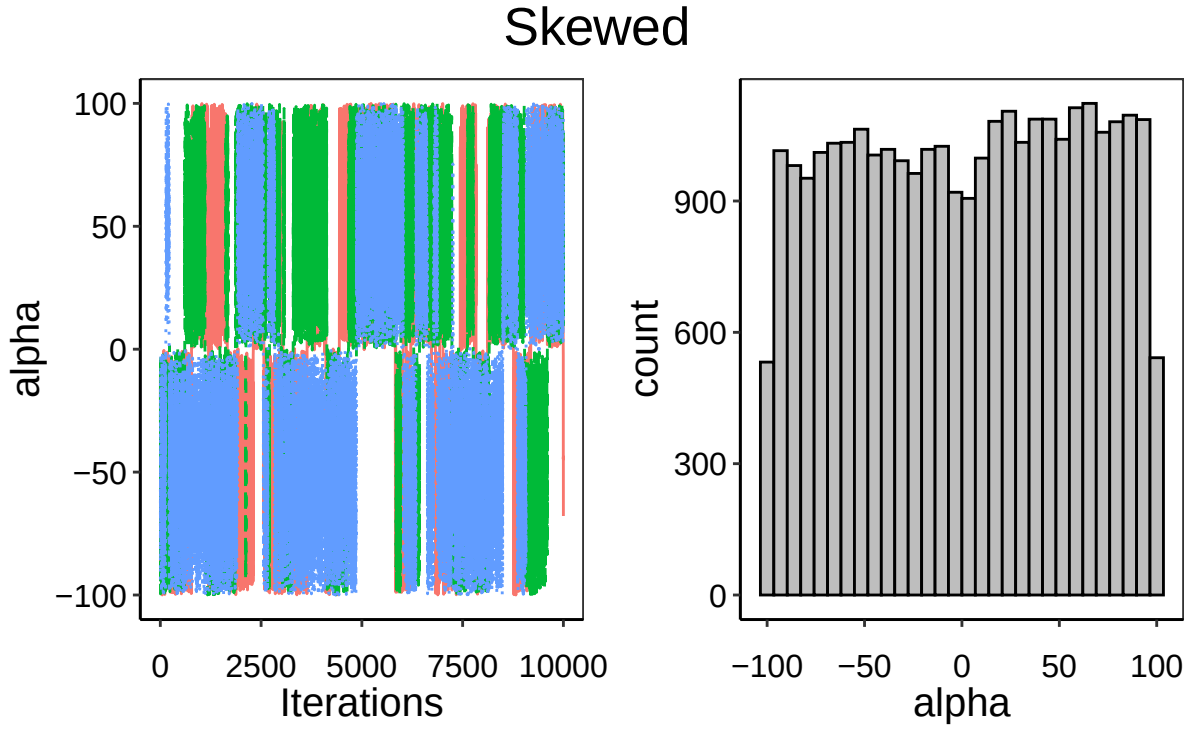


Figure 8: Convergence figures for skew-normal parameters for new source MACT floor analysis for chemical X in subcategory A.

The table below includes all UPL calculations for Example Industry existing and new source MACT floor analyses using a significance of 99% and 3 future test runs.

Table 4: Upper Predictive Limit (UPL) results for existing and new source MACT floors in (measurement units)

HAP	Subcategory	Existing source UPL	New source UPL
Chemical X	Type A	0.6086	0.01577

References

Bhaumik, D.K. and Gibbons, R.D. 2004. An Upper Prediction Limit for the Arithmetic Mean of a Lognormal Random Variable. *Technometrics*, 46(2):239-248.

Dragan Doric, et al. 2007. On measuring skewness and kurtosis. Springer Science + Business Media B. V. 2007.

Appendix A: Existing Source data tables

Example Industry existing source MACT floor analysis data set for HAP *chemical X* in *subcategory A*:

Table 5: Top performing source emissions data used in existing source UPL calculations.

emissions	sources	ln_emiss
0.008808290	A	-4.7320620
0.011979167	A	-4.4245862
0.009326425	A	-4.6749035
0.011052632	B	-4.5050867
0.020769870	B	-3.8742519
0.014084507	B	-4.2626799
0.012000000	B	-4.4228486
0.006000000	B	-5.1159958
0.004000000	B	-5.5214609
0.019000000	B	-3.9633163
0.014000000	B	-4.2686979
0.016000000	B	-4.1351666
0.012000000	B	-4.4228486
0.006000000	B	-5.1159958
0.005000000	B	-5.2983174
0.316000000	C	-1.1520131
0.325000000	C	-1.1239301
0.319000000	C	-1.1425642
0.419587629	D	-0.8684829
0.267010309	D	-1.3204680
0.318041237	D	-1.1455742
0.521265823	E	-0.6514952
0.456532663	E	-0.7840950
0.573500000	E	-0.5559973

Appendix B: NSPS data tables

Example Industry new source MACT floor analysis data set for HAP *chemical X* in *subcategory A*:

Table 6: Best performing source emissions data used in NSPS UPL calculations.

emissions	sources	ln_emiss
0.008808290	A	-4.732062
0.011979167	A	-4.424586
0.009326425	A	-4.674904

Appendix C: R code

Disclaimer

The United States Environmental Protection Agency (EPA) GitHub project code is provided on an “as is” basis and the user assumes responsibility for its use. EPA has relinquished control of the information and no longer has responsibility to protect the integrity, confidentiality, or availability of the information. Any reference to specific commercial products, processes, or services by service mark, trademark, manufacturer, or otherwise, does not constitute or imply their endorsement, recommendation or favoring by EPA. The EPA seal and logo shall not be used in any manner to imply endorsement of any commercial product or activity by EPA or the United States Government.

MACT_EG()

```
#> function (CAA_section = 112, data)
#> {
#>   if (("emissions" %in% names(data)) == FALSE) {
#>     stop("data must have numeric column named 'emissions' ")
#>   }
#>   if (("sources" %in% names(data)) == FALSE) {
#>     stop("data must have character or factor column named 'sources' ")
#>   }
#>   if (!is.numeric(data$emissions)) {
#>     stop("Emissions must be numeric vector")
#>   }
#>   if ((!is.character(data$sources)) & (!is.factor(data$sources))) {
#>     stop("Sources must be a character or factor vector")
#>   }
#>   dat_means = dplyr::summarize(data, means = mean(emissions),
#>     .by = "sources")
#>   n_sources = length(unique(data$sources))
#>   if (CAA_section == 129) {
#>     n_topsources = ceiling(0.12 * n_sources)
#>     set.seed(1)
#>     source_index = sample.int(n_sources, n_topsources)
#>     dat_top = data[source_index, ]
#>   }
#>   else if (CAA_section == 112) {
#>     if (n_sources >= 30) {
#>       n_topsources = ceiling(0.12 * n_sources)
#>       top_list = dat_means[order(dat_means$means, decreasing = F),
#>         ]
#>       top_list = top_list$sources[1:n_topsources]
#>       dat_top = subset(data, data$sources %in% top_list)
#>     }
#>     else if (n_sources < 30) {
#>       n_topsources = 5
#>       top_list = dat_means[order(dat_means$means, decreasing = F),
#>         ]
#>       top_list = top_list$sources[1:n_topsources]
#>       dat_top = subset(data, data$sources %in% top_list)
#>     }
#>   }
#>   dat_topmeans = dplyr::summarize(dat_top, means = mean(emissions),
#>     .by = "sources", counts = dplyr::n())
```

```

#>   dat_topmeans$sources = as.factor(dat_topmeans$sources)
#>   dat_topmeans$sources = forcats::fct_reorder(dat_topmeans$sources,
#>       dat_topmeans$means, .desc = FALSE)
#>   dat_top$sources = factor(dat_top$sources, levels = levels(dat_topmeans$sources))
#>   dat_topmeans = dplyr::arrange(dat_topmeans, means)
#>   dat_top = dplyr::arrange(dat_top, sources)
#>   dat_top$sources = droplevels(dat_top$sources)
#>   return(dat_top)
#> }
#> <bytecode: 0x00000214c1b93eb0>
#> <environment: namespace:UPLforOAR>

```

MACT_NSPTS()

```

#> function (data)
#> {
#>   if (("emissions" %in% names(data)) == FALSE) {
#>     stop("data must have numeric column named 'emissions' ")
#>   }
#>   if (("sources" %in% names(data)) == FALSE) {
#>     stop("data must have character or factor column named 'sources' ")
#>   }
#>   if (!is.numeric(data$emissions)) {
#>     stop("Emissions must be numeric vector")
#>   }
#>   if ((!is.character(data$sources)) & (!is.factor(data$sources))) {
#>     stop("Sources must be a character or factor vector")
#>   }
#>   dat_means = dplyr::summarize(data, means = mean(emissions),
#>       .by = "sources")
#>   top_list = dat_means[order(dat_means$means, decreasing = F),
#>       ]
#>   top_source = top_list$sources[1]
#>   dat_top = subset(data, data$sources == top_source)
#>   dat_top$sources = as.factor(dat_top$sources)
#>   dat_top$sources = droplevels(dat_top$sources)
#>   return(dat_top)
#> }
#> <bytecode: 0x00000214c2777848>
#> <environment: namespace:UPLforOAR>

```

write_likelihood()

```

#> function (distribution, manual_prior = FALSE, write_wd = NULL)
#> {
#>   current_wd = getwd()
#>   if (is.null(write_wd)) {
#>     write_wd = "inst/JAGS/"
#>   }
#>   setwd(write_wd)
#>   if (!manual_prior) {
#>     if (distribution == "Normal") {
#>       JAGS_model = "Emission_normal_JAGS.R"
#>       cat("model {\n      # priors\n      emission_sd~dunif(0.001*sdY,1000*sdY)\n
#>       file = JAGS_model)

```

```

#>     }
#>     else if (distribution == "Lognormal") {
#>         JAGS_model = "Emission_lnorm_JAGS.R"
#>         cat("model {\n      # priors\n          sd_ln ~ dunif( 0.001*sdOfLogY , 1000*sdOfLogY )\n
#>             file = JAGS_model)
#>     }
#>     else if (distribution == "Skewed") {
#>         JAGS_model = "Emission_skewed_JAGS.R"
#>         cat("data {\n          for(i in 1:length(emission_xi)) {\n          zeros[i] <- (
#>             file = JAGS_model)
#>     }
#>     else if (distribution == "Gamma") {
#>         JAGS_model = "Emission_gamma_JAGS.R"
#>         cat("model {\n      # priors\n          rate_em~dunif(0,maxY/(sdY^2)) #must be positive\n
#>             file = JAGS_model)
#>     }
#>     else if (distribution == "Beta") {
#>         JAGS_model = "Emission_beta_JAGS.R"
#>         cat("model {\n      # priors\n          # note that both can be pos or neg, but the minim
#>             file = JAGS_model)
#>     }
#> }
#> else if (manual_prior) {
#>     if (distribution == "Normal") {
#>         JAGS_model = "Manual_emission_normal_JAGS.R"
#>         cat("model {\n      # priors\n          emission_sd~dunif(low1,up1)\n          emission_r
#>             file = JAGS_model)
#>     }
#>     else if (distribution == "Lognormal") {
#>         JAGS_model = "Manual_emission_lnorm_JAGS.R"
#>         cat("model {\n      # priors\n          sd_ln ~ dunif(low1,up1)\n          u_ln ~ dnorm(
#>             file = JAGS_model)
#>     }
#>     else if (distribution == "Skewed") {
#>         JAGS_model = "Manual_emission_skewed_JAGS.R"
#>         cat("data {\n          for(i in 1:length(emission_xi)) {\n          zeros[i] <- (
#>             file = JAGS_model)
#>     }
#>     else if (distribution == "Gamma") {
#>         JAGS_model = "Manual_emission_gamma_JAGS.R"
#>         cat("model {\n      # priors\n          rate_em~dunif(low1,up1) #must be positive\n
#>             file = JAGS_model)
#>     }
#>     else if (distribution == "Beta") {
#>         JAGS_model = "Manual_emission_beta_JAGS.R"
#>         cat("model {\n      # priors\n          # note that both can be pos or neg, but the minim
#>             file = JAGS_model)
#>     }
#> }
#> }
#> setwd(current_wd)
#> }
#> <bytecode: 0x00000214bfa26570>
#> <environment: namespace:UPLforOAR>

```

```
setup_likelihood()
```

```
#> function (distribution, data, manual_prior = FALSE, prior_list = NULL,
#>   random = FALSE)
#> {
#>   JAGS_path = system.file("JAGS", package = "UPLforOAR", mustWork = TRUE)
#>   if (("emissions" %in% names(data)) == FALSE) {
#>     stop("data must have numeric column named 'emissions' ")
#>   }
#>   if (!is.numeric(data$emissions)) {
#>     stop("Emissions must be numeric")
#>   }
#>   mu = mean(data$emissions)
#>   sigma = stats::sd(data$emissions)
#>   if (!manual_prior) {
#>     if (distribution == "Normal") {
#>       JAGS_model = runjags::read.jagsfile(paste0(JAGS_path,
#>         "/Emission_normal_JAGS.R"))
#>       par_list = c("emission_hat", "pdf_obs", "pdf_hat",
#>         "emission_mean", "emission_sd")
#>       data_inits = list(list(.RNG.name = "base::Wichmann-Hill",
#>         .RNG.seed = 5, emission_mean = mu, emission_sd = sigma),
#>         list(.RNG.name = "base::Wichmann-Hill", .RNG.seed = 12,
#>         emission_mean = 1.5 * mu, emission_sd = 0.5 *
#>         sigma), list(.RNG.name = "base::Wichmann-Hill",
#>         .RNG.seed = 151, emission_mean = 0.5 * mu,
#>         emission_sd = 1.5 * sigma))
#>     }
#>     else if (distribution == "Lognormal") {
#>       ln_emiss = log(data$emissions)
#>       ln_mu = mean(ln_emiss, na.rm = TRUE)
#>       ln_sig = stats::sd(ln_emiss, na.rm = TRUE)
#>       JAGS_model = runjags::read.jagsfile(paste0(JAGS_path,
#>         "/Emission_lnorm_JAGS.R"))
#>       par_list = c("emission_hat", "pdf_obs", "pdf_hat",
#>         "u_ln", "sd_ln")
#>       data_inits = list(list(.RNG.name = "base::Wichmann-Hill",
#>         .RNG.seed = 5, u_ln = ln_mu, sd_ln = ln_sig),
#>         list(.RNG.name = "base::Wichmann-Hill", .RNG.seed = 12,
#>         u_ln = 1.5 * ln_mu, sd_ln = 0.5 * ln_sig),
#>         list(.RNG.name = "base::Wichmann-Hill", .RNG.seed = 151,
#>         u_ln = 0.5 * ln_mu, sd_ln = 1.5 * ln_sig))
#>     }
#>     else if (distribution == "Skewed") {
#>       skew1 = min(0.99, abs((1/length(data$emissions)) *
#>         sum(((data$emissions - mu)/sigma)^3)))
#>       delta = sqrt((pi/2) * ((abs(skew1)^(2/3))/((abs(skew1)^(2/3)) +
#>         ((4 - pi)/2)^(2/3))))
#>       delta = delta * abs(skew1)/skew1
#>       alpha = delta/sqrt(1 - delta^2)
#>       omega = sigma/sqrt(1 - 2 * delta^2/pi)
#>       xi = mu - omega * delta * sqrt(2/pi)
#>       JAGS_model = runjags::read.jagsfile(paste0(JAGS_path,
#>         "/Emission_skewed_JAGS.R"))
#>     }
#>   }
#> }
```

```

#>         par_list = c("omega", "xi", "alpha")
#>         data_inits = list(list(.RNG.name = "base::Wichmann-Hill",
#>                               .RNG.seed = 5, xi = xi, omega = omega, alpha = alpha),
#>                            list(.RNG.name = "base::Wichmann-Hill", .RNG.seed = 12,
#>                               xi = 1.5 * xi, omega = 0.5 * omega, alpha = 0.5 *
#>                               alpha), list(.RNG.name = "base::Wichmann-Hill",
#>                               .RNG.seed = 151, xi = 0.5 * xi, omega = 1.5 *
#>                               omega, alpha = 1.5 * alpha))
#>     }
#>     else if (distribution == "Gamma") {
#>         shape = mu^2/sigma^2
#>         rate = mu/sigma^2
#>         JAGS_model = runjags::read.jagsfile(paste0(JAGS_path,
#>            "/Emission_gamma_JAGS.R"))
#>         par_list = c("emission_hat", "pdf_obs", "pdf_hat",
#>            "rate_em", "shape_em")
#>         data_inits = list(list(.RNG.name = "base::Wichmann-Hill",
#>                               .RNG.seed = 5, rate_em = rate, shape_em = shape),
#>                            list(.RNG.name = "base::Wichmann-Hill", .RNG.seed = 12,
#>                               rate_em = 1.5 * rate, shape_em = 0.5 * shape),
#>                            list(.RNG.name = "base::Wichmann-Hill", .RNG.seed = 151,
#>                               rate_em = 0.5 * rate, shape_em = 1.5 * shape))
#>     }
#>     else if (distribution == "Beta") {
#>         if (min(data$emissions) < 0) {
#>             stop("Cannot use beta distribution with emissions less than 0")
#>         }
#>         if (max(data$emissions) > 1) {
#>             stop("Cannot use beta distribution with emissions greater than 1")
#>         }
#>         alpha = mu^2/sigma^2 - mu^3/sigma^2 - mu
#>         beta = mu - 1 + mu/sigma^2 + mu^3/sigma^2 - 2 * mu^2/sigma^2
#>         JAGS_model = runjags::read.jagsfile(paste0(JAGS_path,
#>            "/Emission_beta_JAGS.R"))
#>         par_list = c("emission_hat", "pdf_obs", "pdf_hat",
#>            "alpha_em", "beta_em")
#>         data_inits = list(list(.RNG.name = "base::Wichmann-Hill",
#>                               .RNG.seed = 5, beta_em = beta, alpha_em = alpha),
#>                            list(.RNG.name = "base::Wichmann-Hill", .RNG.seed = 12,
#>                               beta_em = 1.5 * beta, alpha_em = 0.5 * alpha),
#>                            list(.RNG.name = "base::Wichmann-Hill", .RNG.seed = 151,
#>                               beta_em = 0.5 * beta, alpha_em = 1.5 * alpha))
#>     }
#>     if (random) {
#>         data_inits = list(data_inits[[1]][names(data_inits[[1]])] %in%
#>            c(".RNG.name", ".RNG.seed") == FALSE, data_inits[[2]][names(data_inits[[2]])] %in%
#>            c(".RNG.name", ".RNG.seed") == FALSE, data_inits[[3]][names(data_inits[[3]])] %in%
#>            c(".RNG.name", ".RNG.seed") == FALSE])
#>     }
#>     output = list(model_code = JAGS_model, par_list = par_list,
#>         data = data, distribution = distribution, dat_inits = data_inits,
#>         manual_prior = manual_prior)
#> }
#> else if (manual_prior) {

```

```

#> if (is.null(prior_list)) {
#>   stop("Please provide a list of upper and lower bounds for manual priors")
#> }
#> if (distribution == "Normal") {
#>   JAGS_model = runjags::read.jagsfile(paste0(JAGS_path,
#>     "/Manual_emission_normal_JAGS.R"))
#>   par_list = c("emission_hat", "pdf_obs", "pdf_hat",
#>     "emission_mean", "emission_sd")
#>   data_inits = list(list(.RNG.name = "base::Wichmann-Hill",
#>     .RNG.seed = 5, emission_mean = mean(c(prior_list[3],
#>     prior_list[4])), emission_sd = mean(c(prior_list[1],
#>     prior_list[2]))), list(.RNG.name = "base::Wichmann-Hill",
#>     .RNG.seed = 12, emission_mean = 0.9 * prior_list[4],
#>     emission_sd = 0.9 * prior_list[2]), list(.RNG.name = "base::Wichmann-Hill",
#>     .RNG.seed = 151, emission_mean = 1.1 * prior_list[3],
#>     emission_sd = max(1.1 * prior_list[1], 1e-06)))
#> }
#> else if (distribution == "Lognormal") {
#>   JAGS_model = runjags::read.jagsfile(paste0(JAGS_path,
#>     "/Manual_emission_lnorm_JAGS.R"))
#>   par_list = c("emission_hat", "pdf_obs", "pdf_hat",
#>     "u_ln", "sd_ln")
#>   if (prior_list[4] < 0) {
#>     initial1 = 1.1 * prior_list[4]
#>   }
#>   else if (prior_list[4] > 0) {
#>     initial1 = 0.9 * prior_list[4]
#>   }
#>   if (prior_list[3] < 0) {
#>     initial2 = 0.9 * prior_list[3]
#>   }
#>   else if (prior_list[3] > 0) {
#>     initial2 = 1.1 * prior_list[3]
#>   }
#>   data_inits = list(list(.RNG.name = "base::Wichmann-Hill",
#>     .RNG.seed = 5, u_ln = mean(c(prior_list[3], prior_list[4])),
#>     sd_ln = mean(c(prior_list[1], prior_list[2]))),
#>     list(.RNG.name = "base::Wichmann-Hill", .RNG.seed = 12,
#>     u_ln = initial1, sd_ln = 0.9 * prior_list[2]),
#>     list(.RNG.name = "base::Wichmann-Hill", .RNG.seed = 151,
#>     u_ln = initial2, sd_ln = max(1.1 * prior_list[1],
#>     1e-06)))
#> }
#> else if (distribution == "Skewed") {
#>   if (prior_list[4] < 0) {
#>     initial1 = 1.1 * prior_list[4]
#>   }
#>   else if (prior_list[4] > 0) {
#>     initial1 = 0.9 * prior_list[4]
#>   }
#>   if (prior_list[3] < 0) {
#>     initial2 = 0.9 * prior_list[3]
#>   }
#>   else if (prior_list[3] > 0) {

```

```

#>         initial2 = 1.1 * prior_list[3]
#>     }
#>     if (prior_list[6] < 0) {
#>         initial3 = 1.1 * prior_list[6]
#>     }
#>     else if (prior_list[6] > 0) {
#>         initial3 = 0.9 * prior_list[6]
#>     }
#>     if (prior_list[5] < 0) {
#>         initial4 = 0.9 * prior_list[5]
#>     }
#>     else if (prior_list[5] > 0) {
#>         initial4 = 1.1 * prior_list[5]
#>     }
#>     JAGS_model = runjags::read.jagsfile(paste0(JAGS_path,
#>         "/Manual_emission_skewed_JAGS.R"))
#>     par_list = c("omega", "xi", "alpha")
#>     data_inits = list(list(.RNG.name = "base::Wichmann-Hill",
#>         .RNG.seed = 5, xi = mean(c(prior_list[3], prior_list[4])),
#>         omega = mean(c(prior_list[1], prior_list[2])),
#>         alpha = mean(c(prior_list[5], prior_list[6]))),
#>         list(.RNG.name = "base::Wichmann-Hill", .RNG.seed = 12,
#>             xi = initial1, omega = max(1.1 * prior_list[1],
#>                 1e-05), alpha = initial4), list(.RNG.name = "base::Wichmann-Hill",
#>                 .RNG.seed = 151, xi = initial2, omega = 0.9 *
#>                 prior_list[2], alpha = initial3))
#> }
#> else if (distribution == "Gamma") {
#>     if (prior_list[3] <= 0) {
#>         initial2 = 1e-06
#>     }
#>     else if (prior_list[3] > 0) {
#>         initial2 = 1.1 * prior_list[3]
#>     }
#>     if (prior_list[1] <= 0) {
#>         initial1 = 1e-06
#>     }
#>     else if (prior_list[1] > 0) {
#>         initial1 = 1.1 * prior_list[1]
#>     }
#>     JAGS_model = runjags::read.jagsfile(paste0(JAGS_path,
#>         "/Manual_emission_gamma_JAGS.R"))
#>     par_list = c("emission_hat", "pdf_obs", "pdf_hat",
#>         "rate_em", "shape_em")
#>     data_inits = list(list(.RNG.name = "base::Wichmann-Hill",
#>         .RNG.seed = 5, rate_em = mean(c(prior_list[1],
#>             prior_list[2])), shape_em = mean(c(prior_list[3],
#>             prior_list[4])), list(.RNG.name = "base::Wichmann-Hill",
#>             .RNG.seed = 12, rate_em = initial1, shape_em = initial2),
#>             list(.RNG.name = "base::Wichmann-Hill", .RNG.seed = 151,
#>                 rate_em = 0.9 * prior_list[2], shape_em = 0.9 *
#>                 prior_list[4]))
#> }
#> else if (distribution == "Beta") {

```

```

#>         if (min(data$emissions) < 0) {
#>             stop("Cannot use beta distribution with emissions less than 0")
#>         }
#>         if (max(data$emissions) > 1) {
#>             stop("Cannot use beta distribution with emissions greater than 1")
#>         }
#>         if (prior_list[2] < 0) {
#>             initial1 = 1.1 * prior_list[2]
#>         }
#>         else if (prior_list[2] > 0) {
#>             initial1 = 0.9 * prior_list[2]
#>             if (initial1 < (-1)) {
#>                 initial1 = -0.99999
#>             }
#>         }
#>         if (prior_list[1] < 0) {
#>             initial2 = 0.9 * prior_list[1]
#>         }
#>         else if (prior_list[1] > 0) {
#>             initial2 = 1.1 * prior_list[1]
#>             if (initial1 < (-1)) {
#>                 initial1 = -0.99999
#>             }
#>         }
#>         if (prior_list[4] < 0) {
#>             initial3 = 1.1 * prior_list[4]
#>         }
#>         else if (prior_list[4] > 0) {
#>             initial3 = 0.9 * prior_list[4]
#>             if (initial1 < (-1)) {
#>                 initial1 = -0.99999
#>             }
#>         }
#>         if (prior_list[3] < 0) {
#>             initial4 = 0.9 * prior_list[3]
#>         }
#>         else if (prior_list[3] > 0) {
#>             initial4 = 1.1 * prior_list[3]
#>             if (initial1 < (-1)) {
#>                 initial1 = -0.99999
#>             }
#>         }
#>         JAGS_model = runjags::read.jagsfile(paste0(JAGS_path,
#>             "/Manual_emission_beta_JAGS.R"))
#>         par_list = c("emission_hat", "pdf_obs", "pdf_hat",
#>             "alpha_em", "beta_em")
#>         data_inits = list(list(.RNG.name = "base::Wichmann-Hill",
#>             .RNG.seed = 5, beta_em = mean(c(prior_list[3],
#>             prior_list[4])), alpha_em = mean(c(prior_list[1],
#>             prior_list[2])), list(.RNG.name = "base::Wichmann-Hill",
#>             .RNG.seed = 12, beta_em = initial3, alpha_em = initial1),
#>             list(.RNG.name = "base::Wichmann-Hill", .RNG.seed = 151,
#>             beta_em = initial4, alpha_em = initial2))
#>     }

```

```

#>         if (random) {
#>             data_inits = list(data_inits[[1]][names(data_inits[[1]])] %in%
#>                 c(".RNG.name", ".RNG.seed") == FALSE, data_inits[[2]][names(data_inits[[2]])] %in%
#>                 c(".RNG.name", ".RNG.seed") == FALSE, data_inits[[3]][names(data_inits[[3]])] %in%
#>                 c(".RNG.name", ".RNG.seed") == FALSE])
#>         }
#>         output = list(model_code = JAGS_model, par_list = par_list,
#>             data = data, manual_prior = manual_prior, distribution = distribution,
#>             dat_inits = data_inits, prior_list = prior_list)
#>     }
#>     return(output)
#> }
#> <bytecode: 0x00000214c359ec50>
#> <environment: namespace:UPLforOAR>

```

run_likelihood()

```

#> function (model_input, xvals = NULL, maxY = NULL, future_runs = 3)
#> {
#>     future_runs = as.integer(future_runs)
#>     if (!is.integer(future_runs)) {
#>         stop("future_runs must be a positive integer")
#>     }
#>     if (future_runs < 1) {
#>         stop("future_runs must be a positive integer")
#>     }
#>     manual_prior = model_input$manual_prior
#>     data = model_input$data
#>     Sys.setenv(`_R_CHECK_LIMIT_CORES` = FALSE)
#>     verify_install = runjags::testjags(silent = TRUE)
#>     if (!verify_install$JAGS.available) {
#>         stop("Please install JAGS")
#>     }
#>     if (is.null(maxY)) {
#>         maxY = 3 * max(data$emissions)
#>     }
#>     if ((model_input$distribution == "Beta") & (maxY > 1)) {
#>         stop("Cannot use beta distribution with max emissions greater than 1")
#>     }
#>     verify_install$JAGS.available
#>     n.adapt = 10000
#>     n.update = 10000
#>     n.iter = 10000
#>     if (is.null(xvals)) {
#>         xvals = seq(0, maxY, length.out = 1024)
#>     }
#>     if ((model_input$distribution == "Beta") & (max(xvals) >
#>         1)) {
#>         stop("Cannot use beta distribution with max xvals greater than 1")
#>     }
#>     if (!manual_prior) {
#>         data_list = list(emission_xi = data$emissions, n_draws = future_runs,
#>             sdOfLogY = stats::sd(log(data$emissions), na.rm = T),
#>             maxY = maxY, sdY = stats::sd(data$emissions), meanOfLogY = mean(log(data$emissions),

```

```

#>         na.rm = T), pi = pi, x_hat = xvals, n_x_hat = length(xvals))
#>   }
#>   else if (manual_prior) {
#>     if (length(model_input$prior_list) == 4) {
#>       data_list = list(emission_xi = data$emissions, n_draws = future_runs,
#>         sdOfLogY = stats::sd(log(data$emissions), na.rm = T),
#>         maxY = maxY, sdY = stats::sd(data$emissions),
#>         meanOfLogY = mean(log(data$emissions), na.rm = T),
#>         pi = pi, x_hat = xvals, n_x_hat = length(xvals),
#>         low1 = model_input$prior_list[1], up1 = model_input$prior_list[2],
#>         low2 = model_input$prior_list[3], up2 = model_input$prior_list[4])
#>     }
#>     else if (length(model_input$prior_list) == 6) {
#>       data_list = list(emission_xi = data$emissions, n_draws = future_runs,
#>         sdOfLogY = stats::sd(log(data$emissions), na.rm = T),
#>         maxY = maxY, sdY = stats::sd(data$emissions),
#>         meanOfLogY = mean(log(data$emissions), na.rm = T),
#>         pi = pi, x_hat = xvals, n_x_hat = length(xvals),
#>         low1 = model_input$prior_list[1], up1 = model_input$prior_list[2],
#>         low2 = model_input$prior_list[3], up2 = model_input$prior_list[4],
#>         low3 = model_input$prior_list[5], up3 = model_input$prior_list[6])
#>     }
#>   }
#>   }
#>   cl3 = parallel::makeCluster(3)
#>   rjm = suppressWarnings(runjags::run.jags(model = model_input$model_code,
#>     data = data_list, monitor = model_input$par_list, method = "rjparallel",
#>     summarise = FALSE, cl = cl3, n.chains = length(model_input$dat_inits),
#>     burnin = n.update, adapt = n.adapt, sample = n.iter,
#>     inits = model_input$dat_inits))
#>   output = list(run_results = rjm, distribution = model_input$distribution,
#>     manual_prior = manual_prior, data = model_input$data,
#>     xvals = xvals, future_runs = future_runs)
#>   parallel::stopCluster(cl3)
#>   return(output)
#> }
#> <bytecode: 0x00000214b97246d8>
#> <environment: namespace:UPLforOAR>

```

output_likelihood()

```

#> function (jags_model_run, significance = 0.99)
#> {
#>   if (significance >= 1) {
#>     stop("significance must be greater than 0 and less than 1")
#>   }
#>   if (significance <= 0) {
#>     stop("significance must be greater than 0 and less than 1")
#>   }
#>   distribution = jags_model_run$distribution
#>   data = jags_model_run$data
#>   future_runs = jags_model_run$future_runs
#>   xvals = jags_model_run$xvals
#>   if (distribution == "Skewed") {
#>     xi_quant = as.matrix(runjags::combine.mcmc(coda::as.mcmc.list(jags_model_run$run_results,

```

```

#>         vars = "xi"))
#>     omega_quant = as.matrix(runjags::combine.mcmc(coda::as.mcmc.list(jags_model_run$run_results,
#>         vars = "omega")))
#>     alpha_quant = as.matrix(runjags::combine.mcmc(coda::as.mcmc.list(jags_model_run$run_results,
#>         vars = "alpha")))
#>     hat_quant = matrix(nrow = length(xi_quant), ncol = future_runs,
#>         data = NA)
#>     pdf_obs = matrix(ncol = nrow(data), nrow = length(xi_quant),
#>         data = NA)
#>     pdf_hat = matrix(ncol = length(xvals), nrow = length(xi_quant),
#>         data = NA)
#>     for (i in 1:length(xi_quant)) {
#>         set.seed(12)
#>         Fy_sn = sn::dsn(xvals, xi = (xi_quant[i]), omega = (omega_quant[i]),
#>             alpha = (alpha_quant[i]))
#>         set.seed(12)
#>         pdf_obs[i, ] = sn::dsn(data$emissions, xi = (xi_quant[i]),
#>             omega = (omega_quant[i]), alpha = (alpha_quant[i]))
#>         pdf_hat[i, ] = Fy_sn
#>         if (all(Fy_sn == 0)) {
#>             for (k in 1:future_runs) {
#>                 hat_quant[i, k] = NA
#>             }
#>         }
#>         else {
#>             for (k in 1:future_runs) {
#>                 set.seed(12)
#>                 hat_quant[i, k] = sample(x = xvals, size = 1,
#>                     prob = Fy_sn, replace = T)
#>             }
#>         }
#>     }
#>     hat_quant = tibble::as_tibble(hat_quant, .name_repair = "minimal")
#> }
#> else {
#>     pdf_obs = as.matrix(runjags::combine.mcmc(coda::as.mcmc.list(jags_model_run$run_results,
#>         vars = "pdf_obs")))
#>     hat_quant = tibble::as_tibble(as.matrix(runjags::combine.mcmc(coda::as.mcmc.list(jags_model_run$run_results,
#>         vars = "emission_hat"))))
#>     pdf_hat = as.matrix(runjags::combine.mcmc(coda::as.mcmc.list(jags_model_run$run_results,
#>         vars = "pdf_hat")))
#> }
#> names(hat_quant) = sprintf("run%s", seq(1:future_runs))
#> run3_mean = rowMeans(hat_quant)
#> pred_99_3rep = stats::quantile(as.matrix(stats::na.omit(run3_mean)),
#>     probs = c(significance))
#> pdf_hat_quant = matrixStats::colQuantiles(pdf_hat, probs = c(0.5))
#> density_hat = tibble::tibble(pdf_hat = pdf_hat_quant, x_hat = xvals)
#> density_hat = subset(density_hat, is.finite(density_hat$pdf_hat))
#> pdf_obs_quant = tibble::as_tibble(matrixStats::colQuantiles(pdf_obs,
#>     probs = c(0.025, 0.5, 0.975)), .name_repair = "minimal")
#> names(pdf_obs_quant) = c("low", "med", "up")
#> pdf_obs_quant$emissions = data$emissions
#> density_hat$distr = rep(distribution, nrow(density_hat))

```

```

#> pdf_obs_quant$distr = rep(distribution, nrow(pdf_obs_quant))
#> pred_mean = mean(hat_quant$run1, na.rm = TRUE)
#> output = list(predicted_mean = pred_mean, UPL_Bayes = pred_99_3rep,
#>               obs_pdf = pdf_obs_quant, pred_pdf = density_hat, distr = distribution)
#> return(output)
#> }
#> <bytecode: 0x00000214c392d568>
#> <environment: namespace:UPLforOAR>

```

converge_likelihoood()

```

#> function (jags_model_run)
#> {
#>   distribution = jags_model_run$distribution
#>   gelman_list = c()
#>   convYN = c()
#>   if (distribution == "Skewed") {
#>     params_list = c("omega", "xi", "alpha")
#>   }
#>   if (distribution == "Normal") {
#>     params_list = c("emission_mean", "emission_sd")
#>   }
#>   if (distribution == "Lognormal") {
#>     params_list = c("u_ln", "sd_ln")
#>   }
#>   if (distribution == "Gamma") {
#>     params_list = c("rate_em", "shape_em")
#>   }
#>   if (distribution == "Beta") {
#>     params_list = c("alpha_em", "beta_em")
#>   }
#>   for (i in 1:length(params_list)) {
#>     param = params_list[i]
#>     result = coda::gelman.diag(coda::as.mcmc.list(jags_model_run$run_results,
#>           vars = param))
#>     gelman_list[i] = result$psrf[1]
#>     if (result$psrf[1] > 1.2) {
#>       convYN[i] = "No"
#>     }
#>     else if (result$psrf[1] < 1.1) {
#>       convYN[i] = "Yes"
#>     }
#>     else if ((result$psrf[1] > 1.1) & (result$psrf[1] < 1.2)) {
#>       convYN[i] = "Weak convergence"
#>     }
#>   }
#>   results = tibble::tibble(params = params_list, gelman_diag = gelman_list,
#>     convYN = convYN)
#>   distr_names = tibble::tibble(distr = rep(distribution, nrow(results)))
#>   results = cbind(distr_names, results)
#>   return(results)
#> }
#> <bytecode: 0x00000214b6c72ee8>
#> <environment: namespace:UPLforOAR>

```

converge_figs()

```
#> function (distribution, jags_model_run)
#> {
#>   if (distribution == "Normal") {
#>     par_list = c("emission_mean", "emission_sd")
#>   }
#>   else if (distribution == "Lognormal") {
#>     par_list = c("u_ln", "sd_ln")
#>   }
#>   else if (distribution == "Skewed") {
#>     par_list = c("omega", "xi", "alpha")
#>   }
#>   else if (distribution == "Gamma") {
#>     par_list = c("rate_em", "shape_em")
#>   }
#>   else if (distribution == "Beta") {
#>     par_list = c("alpha_em", "beta_em")
#>   }
#>   fig_list = list()
#>   for (i in 1:length(par_list)) local({
#>     i = i
#>     step3 = coda::as.mcmc.list(jags_model_run$run_results,
#>       vars = par_list[i])
#>     step4 = runjags::combine.mcmc(step3)
#>     p1 = ggplot2::ggplot() + ggplot2::geom_line(ggplot2::aes(color = "a",
#>       x = 1:10000, y = as.vector(step3[[1]])), linewidth = 0.5,
#>       linetype = 1) + ggplot2::geom_line(ggplot2::aes(color = "b",
#>       x = 1:10000, y = as.vector(step3[[2]])), linewidth = 0.5,
#>       linetype = 2) + ggplot2::geom_line(ggplot2::aes(color = "c",
#>       x = 1:10000, y = as.vector(step3[[3]])), linewidth = 0.5,
#>       linetype = 3) + mcmc_theme() + ggplot2::xlab("Iterations") +
#>       ggplot2::ylab(par_list[i])
#>     p2 = ggplot2::ggplot() + ggplot2::geom_histogram(ggplot2::aes(step4),
#>       color = "black", fill = "grey") + ggplot2::xlab(par_list[i]) +
#>       mcmc_theme()
#>     p12 = p1 + p2 + patchwork::plot_layout(ncol = 2) + patchwork::plot_annotation(title = distribut
#>       ggplot2::theme(plot.title = ggplot2::element_text(size = 20,
#>       hjust = 0.5))
#>     fig_list[[i]] <- p12
#>   })
#>   return(fig_list)
#> }
#> <bytecode: 0x00000214bf3b3f40>
#> <environment: namespace:UPLforOAR>
```

fit_likelihood()

```
#> Error: object 'fit_likelihood' not found
```

Bayesian_UPL()

```
#> function (distr_list = c("Normal", "Skewed", "Lognormal", "Gamma",
#>   "Beta"), data, future_runs = 3, significance = 0.99, xvals = NULL,
#>   maxY = NULL, convergence_report = FALSE, manual_prior = FALSE,
```

```

#>   prior_list = NULL)
#> {
#>   if (convergence_report == TRUE) {
#>     figs_list = list()
#>   }
#>   mod_output_list = c()
#>   conv_output = tibble::tibble()
#>   if (manual_prior) {
#>     if (length(distr_list) > 1) {
#>       stop("You can only run one distribution at a time if supplying priors manually")
#>     }
#>     distribution = distr_list[1]
#>     mod_bayes = setup_likelihood(distribution = distribution,
#>       data = data, manual_prior = manual_prior, prior_list = prior_list)
#>     mod_run = run_likelihood(model_input = mod_bayes, maxY = maxY,
#>       future_runs = future_runs, xvals = xvals)
#>     manual_prior = mod_bayes$manual_prior
#>     mod_output = output_likelihood(jags_model_run = mod_run,
#>       significance = significance)
#>     mod_fit = fit_likelihood(likelihood_result = mod_output)
#>     mod_output_list[[1]] = mod_fit
#>     mod_converge = converge_likelihood(mod_run)
#>     conv_output = rbind(conv_output, mod_converge)
#>     if (convergence_report == TRUE) {
#>       fig_set = converge_figs(distribution, mod_run)
#>       figs_list[[1]] = fig_set
#>       rm(fig_set)
#>     }
#>     rm(mod_run, mod_output, mod_fit)
#>     gc()
#>   }
#>   if (!manual_prior) {
#>     for (j in 1:length(distr_list)) {
#>       distribution = distr_list[j]
#>       mod_bayes = setup_likelihood(distribution = distribution,
#>         data = data, manual_prior = FALSE)
#>       mod_run = run_likelihood(model_input = mod_bayes,
#>         maxY = maxY, future_runs = future_runs, xvals = xvals)
#>       mod_output = output_likelihood(jags_model_run = mod_run,
#>         significance = significance)
#>       mod_fit = fit_likelihood(likelihood_result = mod_output)
#>       mod_output_list[[j]] = mod_fit
#>       mod_converge = converge_likelihood(mod_run)
#>       conv_output = rbind(conv_output, mod_converge)
#>       if (convergence_report == TRUE) {
#>         fig_set = converge_figs(distribution, mod_run)
#>         figs_list[[j]] = fig_set
#>         rm(fig_set)
#>       }
#>       rm(mod_run, mod_output, mod_fit)
#>       gc()
#>     }
#>   }
#>   if (convergence_report == TRUE) {

```

```

#>     current_wd = getwd()
#>     template_path = system.file("templates", package = "EPA.MACT.floor.UPL",
#>     mustWork = TRUE)
#>     rmarkdown::render(paste0(template_path, "/convergence_template.Rmd"),
#>     output_dir = current_wd, output_file = paste0("Bayesian_UPL_convergence_",
#>     format(Sys.time(), "%m%d%Y-%H%M")))
#>   }
#>   fit_table = tibble::tibble(distr = unlist(lapply(mod_output_list,
#>     "[", "distr")), UPL = (as.numeric(lapply(mod_output_list,
#>     "[", "UPL_Bayes"))), SSE = (as.numeric(lapply(mod_output_list,
#>     "[", "SSE"))), Obs_in_CI = (as.numeric(lapply(mod_output_list,
#>     "[", "good_vals"))), pdf_integral = (as.numeric(lapply(mod_output_list,
#>     "[", "pdf_integral"))))
#>   obs_pdf_dat = tibble::tibble()
#>   for (i in 1:length(distr_list)) {
#>     obs_temp = mod_output_list[[i]]$obs_pdf_dat
#>     obs_pdf_dat = rbind(obs_pdf_dat, obs_temp)
#>   }
#>   pred_pdf_dat = tibble::tibble()
#>   for (i in 1:length(distr_list)) {
#>     pred_temp = mod_output_list[[i]]$xhat_pdf_dat
#>     pred_pdf_dat = rbind(pred_pdf_dat, pred_temp)
#>   }
#>   if (any(conv_output$convYN != "Yes")) {
#>     warning("Some parameters have not converged")
#>   }
#>   return_list = list(fit_table = fit_table, conv_output = conv_output,
#>     obs_pdf_dat = obs_pdf_dat, pred_pdf_dat = pred_pdf_dat)
#>   return(return_list)
#> }
#> <bytecode: 0x00000214c3589970>
#> <environment: namespace:UPLforOAR>

```

obs_density()

```

#> function (data, xvals = NULL, up = Inf, low = 0, kernel = "gamma",
#>   bw = NULL)
#> {
#>   if (is.null(bw)) {
#>     bw = stats::sd(data$emissions) * nrow(data)^(-2/5)
#>   }
#>   if (is.null(xvals)) {
#>     xvals = seq(0, 3 * max(data$emissions), length.out = 1024)
#>   }
#>   if (is.numeric(bw)) {
#>     Obs_onPoint = np::npuniden.boundary(X = data$emissions,
#>       Y = data$emissions, a = low, b = up, proper = TRUE,
#>       kertype = kernel, h = bw)
#>     obs_den_df = np::npuniden.boundary(X = data$emissions,
#>       Y = xvals, a = low, b = up, proper = TRUE, kertype = kernel,
#>       h = bw)
#>   }
#>   else if (is.character(bw)) {
#>     Obs_onPoint = np::npuniden.boundary(X = data$emissions,

```

```

#>         Y = data$emissions, bwmethod = bw, a = low, b = up,
#>         proper = TRUE, kertype = kernel)
#>     obs_den_df = np::npuniden.boundary(X = data$emissions,
#>         Y = xvals, a = low, b = up, proper = TRUE, kertype = kernel,
#>         bwmethod = bw)
#> }
#>     obs_den_df = tibble::tibble(x_hat = xvals, ydens = obs_den_df$f)
#>     Obs_onPoint = tibble::tibble(emissions = data$emissions,
#>         ydens = Obs_onPoint$f)
#>     test_int = sfsmisc::integrate.xy(obs_den_df$x_hat, obs_den_df$ydens)
#>     if (abs(test_int - 1) > 0.05) {
#>         warning("density distribution does not integrate to 1, consider adjusting xvals, bandwidth, ")
#>     }
#>     output = list(Obs_onPoint = Obs_onPoint, obs_den_df = obs_den_df)
#>     return(output)
#> }
#> <bytecode: 0x00000214c1f74580>
#> <environment: namespace:UPLforOAR>

```

Appendix D: System and package versions

```
#> R version 4.4.0 (2024-04-24 ucrt)
#> Platform: x86_64-w64-mingw32/x64
#> Running under: Windows 11 x64 (build 22631)
#>
#> Matrix products: default
#>
#>
#> locale:
#> [1] LC_COLLATE=English_United States.utf8
#> [2] LC_CTYPE=English_United States.utf8
#> [3] LC_MONETARY=English_United States.utf8
#> [4] LC_NUMERIC=C
#> [5] LC_TIME=English_United States.utf8
#>
#> time zone: America/New_York
#> tzcode source: internal
#>
#> attached base packages:
#> [1] grid      stats      graphics  grDevices utils      datasets  methods
#> [8] base
#>
#> other attached packages:
#> [1] UPLforOAR_0.1.0    sfsmisc_1.1-22      np_0.60-18           EnvStats_3.1.0
#> [5] lubridate_1.9.4    forcats_1.0.1        stringr_1.5.2         purrr_1.1.0
#> [9] readr_2.1.5        tidyr_1.3.1          tibble_3.3.0          tidyverse_2.0.0
#> [13] ggplot2_4.0.0      dplyr_1.1.4          htmltools_0.5.8.1
#>
#> loaded via a namespace (and not attached):
#> [1] gtable_0.3.6        xfun_0.53            lattice_0.22-6
#> [4] numDeriv_2016.8-1.1 tzdb_0.5.0           quadprog_1.5-8
#> [7] vctrs_0.6.5         tools_4.4.0          generics_0.1.4
#> [10] stats4_4.4.0        parallel_4.4.0       pkgconfig_2.0.3
#> [13] Matrix_1.7-0        RColorBrewer_1.1-3   S7_0.2.0
#> [16] lifecycle_1.0.4     cubature_2.1.4       compiler_4.4.0
#> [19] farver_2.1.2        MatrixModels_0.5-4   mnormt_2.1.1
#> [22] runjags_2.2.2-5     SparseM_1.84-2       quantreg_6.1
#> [25] yaml_2.3.10         pillar_1.11.1        crayon_1.5.3
#> [28] MASS_7.3-60.2       boot_1.3-30          tidyselect_1.2.1
#> [31] digest_0.6.37       stringi_1.8.7        labeling_0.4.3
#> [34] splines_4.4.0       fastmap_1.2.0        cli_3.6.5
#> [37] magrittr_2.0.4      patchwork_1.3.2      survival_3.5-8
#> [40] sn_2.1.1            withr_3.0.2          scales_1.4.0
#> [43] bit64_4.6.0-1       timechange_0.3.0     rmarkdown_2.30
#> [46] matrixStats_1.5.0   bit_4.6.0            rjags_4-17
#> [49] hms_1.1.3           coda_0.19-4.1        evaluate_1.0.5
#> [52] knitr_1.50          rlang_1.1.6          Rcpp_1.1.0
#> [55] glue_1.8.0          rstudioapi_0.17.1    vroom_1.6.6
#> [58] R6_2.6.1
```